

MoCAP: Mobile Code-As-Policies for Zero-Shot Mobile Manipulation from Passive Human Videos

Abstract—Passive human videos of everyday activity offer robots access to a massive amount of human experience without requiring a demonstration for each task. However, using this experience requires recovering knowledge from unstructured video recordings and adapting it to a different body (i.e., the robot). To overcome these challenges, we present mobile code-as-policy (MoCAP), an agentic framework that turns passive human video recordings into guidance for an end-to-end mobile manipulation policy. MoCAP extracts spatial memory and cues of how people prepare to interact with objects, which inform navigation and the search for a manipulation-ready configuration. Guided by passive human experience and current observations, MoCAP uses a closed-loop code-as-policy to coordinate planning and tool use primitives to solve motions under the robot’s geometry and kinematic constraints. On a real mobile manipulator, MoCAP achieves 80% success on each of two mobile manipulation tasks in an indoor environment whereas existing approaches cannot succeed at the task. Video-derived spatial memory improves object-navigation success from 70% to 100%, and human approach prior raises the rate of finding a reachable base pose from 0% to 100% at an unfavorable start location.

I. INTRODUCTION

Household robots should be able to assist different users in a wide range of homes, responding to everyday requests without requiring users to specify every action. Open-vocabulary mobile manipulation moves toward this goal by enabling robots to act on natural-language instructions involving diverse objects and environments [1]. To fulfill a request such as “bring the cup back,” a robot must determine which objects and places are relevant, locate them, and decide how to act. Supporting this ability across homes and users motivates learning from experience beyond the robot’s own interactions.

Passive recordings of human activity offer a potentially abundant source of such experience. Wearable cameras can capture how people interact with objects and their surroundings during everyday life, without requiring robot teleoperation or prescribed demonstrations [2]. We investigate how robots can draw on this accumulated human experience for open-vocabulary mobile manipulation. We consider the setting where a collection of human activity videos is passively recorded through people’s daily lives capturing diverse activities that may be requested of a robot. Rather than reproducing a particular demonstration, the robot must use relevant experience from the collection to guide its actions in its current environment.

However, passive recordings of egocentric human data do not directly provide the information a robot needs to act. Specifically, using passive human experience for robots presents three challenges.

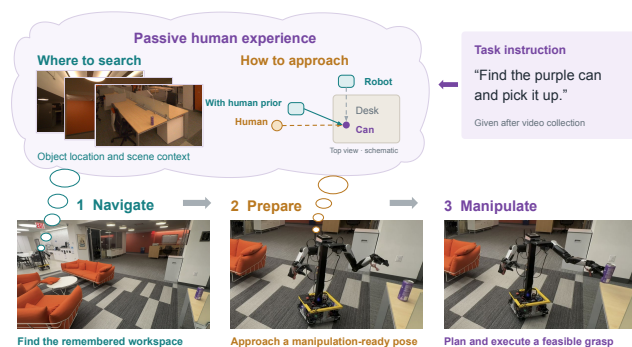


Fig. 1. **MoCAP turns passive human video into guidance for real-world mobile manipulation.** A person goes about everyday activities while a camera records. Later, the robot receives a request: “Find the purple can and pick it up.” MoCAP draws on that earlier experience to carry out the request, from finding the object to getting to a manipulation ready position, then to successfully manipulating, without a demonstration specifically collected for the requested task.

First, the recorded experience lacks explicit spatial structure. A video supplies neither object coordinates nor labeled places and traversable connections. Relevant observations are distributed over time, with objects and regions reappearing from different viewpoints. Searching beyond the current view requires connecting objects to their spatial context.

Second, human and robot sensing and morphology differ. Camera height and viewing angle change the visible scene, while different cameras introduce further differences in intrinsics, field of view, resolution, and color response. These visual discrepancies complicate matching recorded experience to the robot’s current observations. Body dimensions, joint structure, and available motions also determine which actions can be executed: a person can turn sideways through a narrow passage or manipulate an object with a dexterous hand, whereas a mobile manipulator has a different footprint and gripper. Transferring experience therefore requires interpreting observations across cameras and viewpoints and grounding actions in the robot’s own geometry and kinematics.

Third, navigation and manipulation are not explicitly separated in human activity. People naturally adjust their approach and position as they get ready to interact, without marking when navigation ends and manipulation begins. For a robot, reaching the vicinity of an object does not ensure that it can manipulate it. The recording does not explicitly reveal which movements simply bring a person closer and which make the subsequent interaction possible.

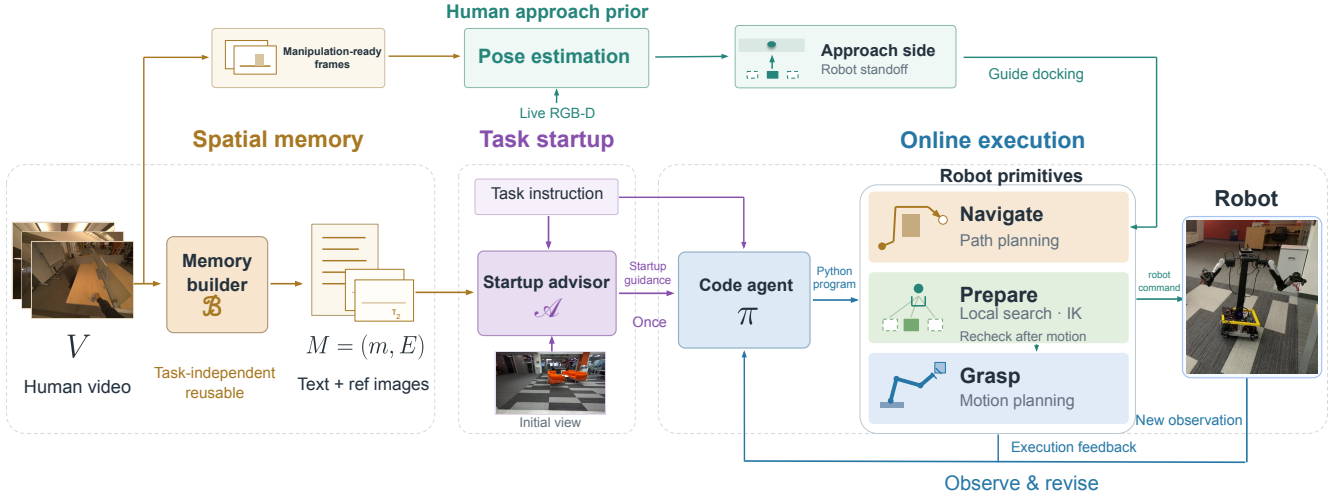


Fig. 2. **MoCAP overview.** Passive human video provides reusable spatial memory and observations of how people prepare to manipulate. At task startup, an advisor VLM relates the spatial memory to the instruction and initial robot view. During execution, the system retrieves a recorded interaction with the target object and estimates where the person in the video stood relative to it from the frame preceding the reach. A closed-loop code policy coordinates navigation and manipulation through planning primitives that check actions against live geometry and robot constraints, revising execution from observations and feedback.

To overcome these challenges, we present **MoCAP**, which turns passive human video into guidance for mobile navigation and manipulation (Fig. 1). MoCAP extracts spatial memory about a place together with records of where people stood when they reached for objects. During task execution, the spatial memory helps locate the requested object, and the human approach prior guides the robot to approach the object in the right way. A code agent uses this guidance together with current observations to coordinate navigation and manipulation primitives. The primitives solve and check motions for the robot’s embodiment, returning execution feedback that lets the agent revise subsequent actions.

Our contributions are:

- **A unified Code-as-Policy system for mobile manipulation.** MoCAP flexibly coordinates navigation and manipulation through executable code, without a predefined division into navigation and manipulation stages. The agent selects and revises actions using prior human experience, live observations, and feedback from robot-grounded primitives.
- **Reusable spatial memory from passive human video.** We develop a fine-grained spatial memory pipeline that draws on the abundance of everyday human activity recordings to extract object relations, landmarks, and place connectivity for subsequent robot search and navigation. This video is task-independent and can be reused to different tasks.
- **Human-guided base placement for manipulation.** We infer the direction to approach object from how a person positioned themselves before manipulation in passive human videos, then search nearby base poses under the robot’s own reachability and collision constraints. A poor arrival side can leave no feasible pose within reach of local adjustment, and this guidance is designed

to turn such arrivals into graspable configurations, using only video that accumulates from everyday activity.

We evaluate MoCAP on a real mobile manipulator in an indoor environment through an object-navigation task and two mobile manipulation tasks. MoCAP achieves 80% success on each manipulation task, whereas existing methods [3] cannot succeed at the task. To examine the contributions of video-derived guidance, we separately evaluate spatial memory for object search and human approach priors for base placement. Spatial memory increases navigation success from 70% to 100%, while controlled comparisons suggest that human approach priors improve manipulation readiness from unfavorable initial base configurations. Base placement guided by the human prior found a reachable pose in 5 of 5 trials whereas a naive proximity-based docking failed in all five. Together, these results suggest that passive human video can help robots complete mobile manipulation tasks by guiding both object search and the approach to manipulation.

II. RELATED WORK

A. Human Video as Scene and Interaction Experience

Using past observations to guide future action is a central motivation for persistent scene representations. Spatial-semantic maps and scene graphs associate objects with locations, appearance, and spatial relations, supporting queries and planning beyond the robot’s current view [4]–[6]. More recently, recorded video has provided another source of scene experience: relational video memories support later scene questions [7], while tour-conditioned navigation retrieves goal views for graph-based execution [8] or generates route instructions for a downstream navigator [9]. Together, these approaches establish how earlier observations can help an agent determine where to search. For mobile manipulation, however, locating the relevant workspace is only part of the

problem: the robot must also approach it from a configuration that permits interaction.

Human activity video offers complementary information about how interaction is prepared and performed. Prior work transfers predicted hand trajectories into robot actions [2], or extracts contact locations and subsequent motion as visual affordances for robot learning [10]. SafeMimic adapts demonstrated actions through safety-checked search [11], and HoMMI learns hand-eye trajectories realized by whole-body control [12]. These approaches motivate using human behavior as more than a record of object locations. MoCAP focuses on a complementary cue: the position from which a person prepares to interact. It combines this cue with spatial memory from the same passive recordings, using scene experience to guide search and pre-contact positioning to guide approach. The recordings are processed before downstream requests are supplied and can support multiple tasks without a prescribed demonstration of each requested robot behavior.

B. Language-Guided Mobile Manipulation

Language-guided robot systems connect high-level requests to executable behavior through perception and action interfaces. Skill-based approaches use language or vision-language models to select and parameterize robot capabilities [13], [14]; Code as Policies extends this interface to executable programs that compose perception and control APIs into executable programs [15]. Incorporating execution feedback enables these systems to revise plans and recover from failures [16], [17], while ASPIRE turns validated program repairs into reusable skills [18]. For mobile manipulation, integrated systems bring object search, navigation, and manipulation into a common execution pipeline [1], [19].

CaP-X provides a closely related foundation for composing perception and control primitives, with its real-robot evaluation centered on tabletop manipulation [3]. Its mobile manipulator interface selects base poses by geometric heuristics and relies on the simulator’s planner to reject infeasible ones. MoCAP adopts the CaP-X program interface but grounds base motion in the real robot: docking goals are validated against the online costmap and executed by a classical navigation stack [20], and arrival is verified with fresh perception before manipulation proceeds. Moving to mobile manipulation also introduces a coupled problem beyond collision-free motion: the robot must decide where to search and from which side to approach for manipulation. A code-based control interface enables these decisions but does not itself supply the scene and interaction experience needed to guide them, failing the tasks. MoCAP draws this guidance from passive human video, using spatial memory to guide search and pre-contact positioning to guide approach, while robot-specific primitives determine executable motions.

C. Base Placement for Mobile Manipulation

Whether a mobile manipulator can execute a manipulation task heavily depends on where it stops. Classical methods precompute a voxelized map of the arm’s reachable

workspace [21] and invert it to obtain the base poses from which a target is reachable [22]. These maps depend only on kinematics, so they are nearly symmetric around the target and cannot tell which side of a table is free or which way a door opens. Learning-based methods recover that context from successful rollouts [23], reinforcement learning with reachability prior [24], at the cost of robot trials in each new environment. Welschehold et al. [25] initialized base poses from observed human torso poses, yet their demonstrations are specific to target task. We instead use passive egocentric video, detecting the transition from walking to reaching and using the recorded human standing position as a good prior to order the search over base poses.

III. METHOD

A. Overview

MoCAP takes a language instruction q , a prior human RGB recording $V = \{I_\tau^h\}$, and live robot observations $o_t = (I_t, D_t, x_t, z_t)$ containing RGB, depth, base pose, and joint state. Camera calibration, robot kinematics, and online local costmaps are available; the recording supplies no metric camera poses or target coordinates. All learned components remain frozen.

Video processing produces reusable spatial memory and records of human interactions. At task startup, an advisor uses the memory, instruction, and initial robot view to provide guidance to a code agent. During execution, the code agent generates short Python programs that invoke robot-grounded primitives and revises subsequent programs using fresh observations and execution feedback. The human preparation prior guides docking, while the primitives resolve motions using live geometry and robot constraints (Fig. 2).

B. Offline Spatial and Interactive Memory Construction

An offline builder organizes the recording into text m and timestamped reference frames E :

$$M = (m, E) = \mathcal{B}(V), \quad E = \{(\tau_k, I_{\tau_k}^h)\}_{k=1}^K. \quad (1)$$

The builder uses GPT-5.6 [26] to generate scene descriptions m that cite relevant video timestamps, then extracts the corresponding reference frames E . Together, the text and frames form the spatial memory M . The text retains places, connections, landmarks, and object descriptions with support surfaces and relative locations. The builder links small-object observations to their surrounding regions and records transition cues: where to turn, which landmark should become visible next, and how to recognize a revisited place. Ambiguous identities and inferred connections remain marked as uncertain. Up to 32 frames are extracted at timestamps cited by the memory, providing visual references for these descriptions.

Given the instruction and initial robot image, a single startup advisor computes $a_0 = \mathcal{A}(M, q, I_0)$. It receives the memory text and reference images with historical and live views explicitly distinguished. Its output describes the robot’s likely location, two to four navigation subgoals with progress cues, and the target’s appearance or support surface. This

guidance is added to the code agent’s initial context and remains available through conversation history. The advisor is called once per task; the agent uses subsequent observations to revise how it follows the guidance.

A separate event set $\mathcal{H}$ records human interactions and the views preceding them. Mean grayscale frame differences identify low-motion windows, sampled at 2 Hz for VLM labeling. For each interaction, the model identifies the object, hand, last frame before reaching, and first contact. We save the ready frame, a view 2 s earlier, and the contact frame. The ready view captures the person’s position before arm motion changes the view or occludes the target; the preceding view provides additional scene context for geometric association. Both memory and event extraction precede downstream instructions.

C. Code-as-Policy and Robot-Grounded Primitives

We implement closed-loop control through a Code-as-Policy module, in which a vision-language code agent generates Python programs that invoke robot-grounded primitives. We denote the agent’s code-generation policy by π . Given the instruction q , startup guidance a_0 , current visual observation and state summaries $\tilde{o}_t$, and conversation history h_t , the agent generates a program c_t , which the runtime executes:

$$c_t = \pi(q, a_0, \tilde{o}_t, h_t), (r_t, o_{t+1}) = \text{ExecuteObserve}(c_t). \quad (2)$$

Execution feedback r_t and a fresh observation inform the next code-generation step. Failed action calls interrupt the remaining program, allowing the agent to revise its plan. History persists in the conversation. A typical manipulation sequence docks to a visible object, prepares the selected arm, samples a checked grasp, and requests its execution. If preparation fails, the subsequent grasp call is skipped, and the agent can change its approach using the returned diagnostics.

SAM 3 [27] masks and live depth locate named targets. The docking primitive converts a visible target into a ground-plane standoff goal facing the object; Nav2 [20] handles navigation, and fresh perception verifies arrival. Bounded turning, forward/backward motion, and lateral translation support search and recovery when no docking target is available. These primitives ground video-derived guidance in the robot’s own viewpoint and motion capabilities.

For manipulation, GraspGen-X [28] proposes grasps for the segmented target. Calibrated transforms convert model grasps to tool center point (TCP) goals, and open-gripper swept-volume checks reject approaches obstructed by observed non-target geometry. The following stages determine where the robot should stand and which grasps it can execute.

D. Approaching Object under Guidance from Human Prior

At docking, we retrieve the first event in $\mathcal{H}$ sharing a non-article word with the target description. Pretrained VGGT [29] processes the robot RGB image together with the event’s preceding and ready views. It estimates camera intrinsics and relative poses from these images, allowing the human and robot cameras to have different parameters

without requiring calibration of the historical recording. Let (R_h, t_h) map robot-camera coordinates to the estimated ready-view camera, and $\tilde{D}_t$ denote predicted robot-view depth. Live depth supplies metric scale:

$$\alpha = \text{median}_{p \in \Omega} \frac{D_t(p)}{\tilde{D}_t(p)}, \quad C_h = -\alpha R_h^\top t_h, \quad (3)$$

where Ω contains valid, confident depth pixels. Live depth is resized to the predicted depth’s image-content region, excluding padding and measurements outside 0.2–8.0 m. We reject estimates when the ratio of the 75th to the 25th percentile of depth ratios exceeds 2.0, or the inferred human camera height lies outside 1.2–2.1 m. Missing events or rejected estimates revert to the original arrival direction.

We project camera center C_h and its optical axis onto the online ZED ground plane, then use the current camera pose in odometry coordinates to obtain horizontal position p_h and heading θ_h . We approximate the person’s stance as $p_s = p_h - \delta e(\theta_h)$, where $e(\theta) = (\cos \theta, \sin \theta)^\top$ and $\delta = 0.30$ m. For target position p_o , the approach bearing is

$$\beta_h = \begin{cases} \theta_h + \pi, & \|p_h - p_o\| \leq \varepsilon, \\ \text{atan2}((p_s - p_o)_y, (p_s - p_o)_x), & \text{otherwise,} \end{cases} \quad (4)$$

with $\varepsilon = 0.30$ m. Near the object, heading avoids a bearing dominated by small position errors. This is a stance heuristic; robot reachability is checked subsequently.

Human and robot reach differ, so we transfer the approach direction while using a robot-specific standoff ρ ; local grasp and IK checks determine the final base configuration. The desired robot camera position is $p_o + \rho e(\beta_h)$, facing the target. The camera-to-base offset determines the chassis goal. Navigation failures trigger attempts at $\beta_h \pm 45^\circ$, followed by the original arrival bearing; blocked starts return recovery feedback. Even when already near the target, a bearing error above 45° triggers repositioning. The suggested arm follows the target’s side relative to the robot, with the human hand breaking ties near the midline. Thus, the prior guides the starting point of preparation, while robot geometry determines distance and arm choice.

E. Local Preparation and Grasp Execution

The preparation primitive tests nearby base configurations before moving. Gripper collision and IK checks screen these configurations before whole-arm trajectory planning, avoiding a full trajectory-planning query for every virtual base pose. It requires a visible target within 0.8 m and evaluates up to 64 proposed grasps. Its $7 \times 7 \times 7$ grid of planar adjustments $u = (\Delta f, \Delta l, \Delta \psi)$ spans forward offsets from -0.10 to 0.20 m and lateral offsets from -0.15 to 0.15 m in 0.05 m steps, with yaw from -15° to 15° in 5° steps. Including zero motion lets an already suitable pose pass directly to grasping. We use T_{XY} to map coordinates from frame Y to frame X . Let C_0 and C_u denote the current and candidate camera frames, and $H(u) = T_{C_0 C_u}$ the induced camera motion, including the camera offset from the chassis rotation center. A grasp pose $G_j = T_{C_0 G_j}$ becomes the candidate TCP goal

$$T_{A,j}(u) = T_{AC} H(u)^{-1} G_j T_{GE}, \quad (5)$$

where T_{AC} is the fixed camera-to-arm-base calibration for the selected arm, and T_{GE} is the TCP frame E expressed in the grasp-model frame G . Thus, $H(u)^{-1}$ expresses the grasp in the candidate camera frame before conversion to the arm-base frame.

Workspace filtering and a cheap geometric score shortlist up to 128 base poses and four grasps per pose for Mink-based IK [30]. A base pose is preparation-ready when at least one grasp passes the gripper collision checks and nominal IK; whole-arm trajectory feasibility is checked later. Small translation and yaw perturbations provide a soft robustness preference. Let $n(u)$ count feasible grasps, $\bar{r}(u)$ be the capped perturbation-convergence count, and $j(u)$ the preferred grasp's joint travel. We select among evaluated poses by

$$c(u) = \frac{\sqrt{\Delta f^2 + \Delta l^2}}{s_t} + \frac{|\Delta \psi|}{s_\psi}, \quad (6)$$

$$u^* = \arg \min_{u: n(u) \geq 1} (\lfloor c(u) \rfloor, -n(u), -\bar{r}(u), j(u), c(u)),$$

with $s_t = 0.02$ m and $s_\psi = 2^\circ$; a workspace score breaks remaining ties. This ordering first favors small base adjustments, then the number and robustness of available grasps within a motion tier. Queries proceed by increasing motion-cost tier. Once the nearest tier containing a feasible pose is fully evaluated within the scheduled shortlist, nominal search stops. The configured IK budget is 15 s, with at most 512 nominal queries and 1024 queries including robustness checks. Perturbations of ± 1 cm along each planar axis and $\pm 1^\circ$ in yaw assess sensitivity to arrival error.

The selected adjustment is executed as an absolute planar base goal. If motion is refused, the primitive tries ranked alternatives within its configured limits. After motion, it checks grasps again using fresh observations at the achieved pose, since controller error can invalidate the virtual prediction. Selecting the exact current pose permits reuse of the existing evaluation. Preparation stores the checked grasp set with the selected arm, base pose, joint state, and timestamp. Subsequent sampling checks its age, robot state, and target consistency before reuse, keeping preparation and grasp execution tied to the same target and feasible alternatives.

cuRobo [31] then uses fresh scene geometry and measured joints to plan whole-arm motion through a pre-grasp pose and constrained approach, considering the retained alternative goals. Planning or execution failure returns control to the agent.

Spatial memory guides where to search, and human approach prior guides base placement for manipulation. The code agent integrates this guidance with live observations and execution feedback, while robot-specific checks and planning determine the motions to execute.

IV. EXPERIMENTS

Our experiments evaluate MoCAP on a real mobile manipulator in open-vocabulary manipulation tasks. We ask three questions: (i) does MoCAP complete tasks that existing modular systems fail to complete? (ii) does the scene memory

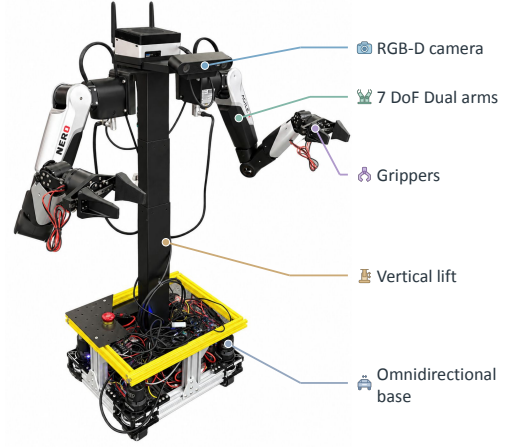


Fig. 3. YOR robot platform used in our experiments, equipped with an RGB-D camera, dual arms with grippers, a vertical lift, and an omnidirectional base.

built from human video reduce the search required to locate an object? (iii) does the human approach prior for base placement increase the chance of reaching a manipulation-ready base pose?

A. Experimental Setup

Experiments use the YOR platform [32] shown in Fig. 3. The robot comprises an omnidirectional base, a vertical lift, two AgileX Nero arms with grippers, and a ZED2i stereo RGB-D camera. A Jetson Orin NX hosts the agent runtime and perception/planning services, with Raspberry Pi services controlling the base and arms. The code agent calls GPT-5.6 [26] with the reasoning effort (thinking level) set to medium. Figure 4 shows the indoor environment, navigation to a trash bin, and two mobile manipulation tasks.

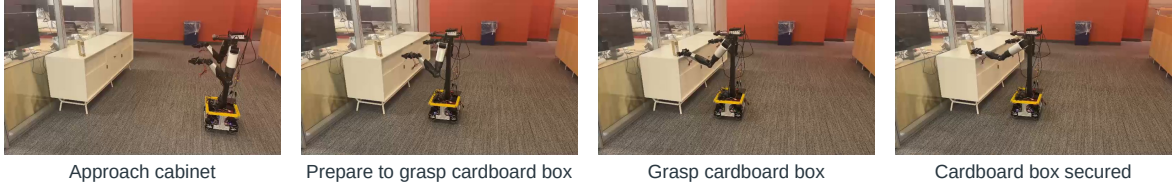
Human activity videos are recorded using Aria Gen 2 glasses [33]. We use only the recorded egocentric RGB video; no other data provided by the glasses are used. GPT-5.6 performs offline video understanding and spatial-memory construction before downstream instructions are supplied, and the same memory is reused across tasks. We evaluate one object-navigation task, navigating to a blue trash bin, and two mobile manipulation tasks, grasping a cardboard box on a white storage cabinet and grasping a can on a table.

Comparisons use matched tasks, object placements, and robot starts, with scene resets between trials. Model weights, prompts, perception backends, and primitive settings remain fixed except for the interface under study. Complete-task outcomes are judged from recorded physical execution. We report successes/trials and success rate (SR), with unsuccessful episodes retained in the denominator. For navigation, we additionally report the numbers of episodes completed successfully within 180 and 300 seconds to evaluate how quickly the robot could navigate.

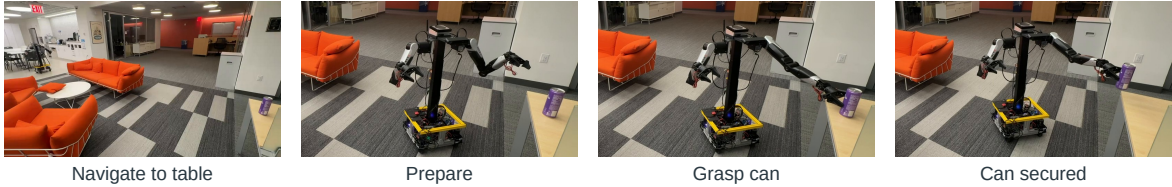
B. Complete Mobile Manipulation

We evaluate MoCAP on two tasks: grasping a cardboard box and grasping a can, each illustrated in Fig. 4(a) and

(a) Grasp cardboard box



(b) Grasp can



(c) Navigate to trash bin

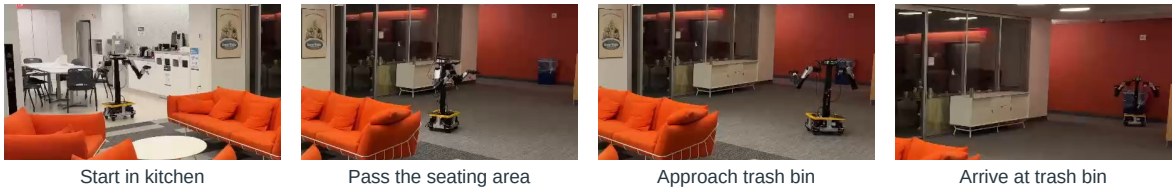


Fig. 4. Example executions of (a) grasping a cardboard box, (b) grasping a can, and (c) navigating to a blue trash bin. Keyframes in each row follow one continuous execution from left to right.

(b). The robot approaches the target, prepares a grasping configuration, and acquires the object. The box task starts with the target in view, providing a simpler setting for evaluating approach and grasping. The can is outside the robot’s initial view, so the robot must first locate it before preparing to grasp it. Success requires grasping the requested object and keeping it securely held.

We compare MoCAP with CaP-X [3], an agentic framework for robotic manipulation. We run it through its mobile manipulator (R1Pro) API, which exposes SAM3 and Contact-GraspNet [34] perception, base navigation and in-place search, IK, and grasping, ported to our robot with the same task instructions, policy model, time limit, and program budget as MoCAP. Base goals are executed by the platform’s base controller, and the operator stops the robot before a collision, which counts as a failure.

Table I reports physical task outcomes. MoCAP succeeds in 4/5 trials (80%) on each task, whereas CaP-X succeeds in 0/5 on both, failing before any grasp attempt. CaP-X’s navigation code assumes the simulator’s map: its standing pose is a geometric heuristic whose collisions the simulator’s planner would reject, so on the real robot the goal falls behind or onto the cabinet in every box trial and the robot had to be stopped before contact. On the can grasping, which starts out of view, its only search is in-place rotation, which never brought the target in the view (0 detections in 144 queries).

C. Spatial Memory and Object Navigation

We evaluate the navigation ability of MoCAP with a task where the robot needs to navigate to a blue trash bin from a

TABLE I
MOBILE MANIPULATION RESULTS (FIVE TRIALS PER TASK).

Task	Method	Successes/trials	SR (%)
Grasp box	MoCAP (ours)	4/5	80
	CaP-X [3]	0/5	0
Grasp can	MoCAP (ours)	4/5	80
	CaP-X [3]	0/5	0

place where it can’t see the target, as illustrated in Fig. 4(c).

Table II compares MoCAP, a no-prior variant, and two external baselines on navigation to the blue trash bin from the kitchen start. CoW [36] selects frontiers geometrically and uses open-vocabulary localization only to decide when the target is found. ApexNav [35] scores frontiers by semantic reasoning and falls back to geometry when semantic cues are weak. We chose these two because they span the range of exploration strategies available without an offline prior. Neither can consume such a prior, so we compare them against the no-prior variant, which operates under the same assumption, and reserve the comparison with full MoCAP for measuring the prior itself. We run ten episodes per method. An episode succeeds if the robot’s base stops within 1m of the blue trash bin and the bin is visibly identified in the terminal robot view. We also report success within fixed time budgets because the prior is intended to improve efficiency in addition to success rate.

MoCAP succeeds in 10/10 episodes, compared with 7/10 for the no-prior variant, 2/10 for ApexNav, and 0/10 for CoW. Within 180 seconds the respective counts are 6, 4, 1,

TABLE II
BLUE-TRASH-BIN NAVIGATION FROM THE KITCHEN.

Method	Successes/trials	Success within 180 s	Success within 300 s
MoCAP (ours)	10/10	6/10	9/10
MoCAP (no prior)	7/10	4/10	6/10
ApexNav [35]	2/10	1/10	2/10
CoW [36]	0/10	0/10	0/10

and 0, and within 300 seconds they are 9, 6, 2, and 0. The gap between MoCAP and the no-prior variant is consistent with a benefit from prior guidance in this setting. The external baselines fail in characteristic ways. CoW consistently stops at a white trash bin in the kitchen that carries a blue notice on its side. Its localizer assigns high confidence to this distractor, and since CoW terminates once confidence exceeds the threshold, it never leaves the starting room. ApexNav fails in two ways. In some episodes it detects the blue bin from a distance but continues exploring instead of committing to it, so the remaining navigation exceeds the time budget. In the others it settles on a small black shredder in an adjacent room and declares the goal reached.

D. Effect of Human Approach Prior in Base Placement

As the robot approaches an object, its direction of arrival shapes the arm’s reach and the local adjustments needed for grasping. The human’s position before interaction provides a reference for this approach. We examine how this guidance affects physical grasping within a fixed time budget and the preparation effort accumulated during execution.

The robot starts from A, B, or C in Fig. 5; the human interaction occurs on the C side. From each matched start, we compare with and without the human approach prior. Each trial executes a fixed sequence of docking followed by local preparation, without the code agent, so neither condition can recover from a failed approach or retry from a different direction. Both conditions share the same perception, docking primitive, local search, and grasp planner. Only the approach direction supplied to docking changes: with the prior it follows the recorded human stance, and without it the robot retains its arrival direction.

Success denotes readiness, defined as the existence of a base pose from which at least one proposed grasp passes the gripper collision checks and nominal IK. We report five trials per start and condition and classify each failure by the stage at which it occurred.

Table III reports the results. With the prior, readiness is reached in 38/45 trials, compared with 18/45 without it. The effect is concentrated at A, where the prior changes 0/15 to 14/15, while C shows no advantage. This follows the layout in Fig. 5: the recorded interaction occurs on the C side, so the arrival direction without the prior already coincides with the human stance there, and A is the start for which the two most disagree. The pooled comparison is therefore not the quantity of interest, and at this sample size it is not significant.

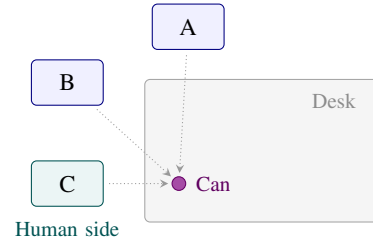


Fig. 5. Schematic layout of the base placement study. A, B, and C denote the three starting configurations around the desk, and the dotted lines indicate the direction to the target can. The recorded human interaction occurs on the C side.

TABLE III
MANIPULATION READINESS (FIFTEEN TRIALS PER START AND CONDITION).

Start	Success rate (%)		Δ
	Without prior	With prior	
A	0.0	93.3	+93.3
B	26.7	80.0	+53.3
C	93.3	80.0	-13.3
All	40.0 (18/45)	84.4 (38/45)	+44.4

V. LIMITATIONS AND CONCLUSION

We presented MoCAP, which turns task-independent passive human video into guidance for training-free OVMM. The video supplies where to search, from a spatial memory of the recorded environment, and from which side to approach, from where a person stood before reaching for an object. Combined with live perception and robot-specific planning, this guidance let a real mobile manipulator complete tasks that a tabletop coding agent could not.

Our evaluation is limited to one environment, one video recording, and three target objects with five to ten trials per condition. The results therefore establish feasibility rather than generality. Whether memory and approach priors transfer across rooms, recordings, and objects absent from the recording remains open. Several design limitations also remain. A fixed startup summary can omit relevant objects or become stale. Local base search depends on visible geometry and the grasp model and does not optimize future visibility. Partial depth observations and imperfect footprint models limit collision checks. Manipulation is limited to grasping and lifting. Future work could evaluate the priors across multiple homes and recordings, and use human video to guide richer skills involving sustained contact, precise placement, or coordinated use of both arms.

REFERENCES

- [1] S. Yenamandra, A. Ramachandran, K. Yadav, A. S. Wang *et al.*, “HomeRobot: Open-vocabulary mobile manipulation,” in *Conference on Robot Learning (CoRL)*, 2023. [Online]. Available: <https://proceedings.mlr.press/v229/yenamandra23a.html>
- [2] H. Bharadhwaj, A. Gupta, S. Tulsiani, and V. Kumar, “Zero-shot robot manipulation from passive human videos,” *arXiv preprint arXiv:2302.02011*, 2023. [Online]. Available: <https://arxiv.org/abs/2302.02011>

- [3] L. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue *et al.*, “CaP-X: A framework for benchmarking and improving coding agents for robot manipulation,” *arXiv preprint arXiv:2603.22435*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.22435>
- [4] C. Huang, O. Mees, A. Zeng, and W. Burgard, “Visual language maps for robot navigation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.05714>
- [5] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula *et al.*, “ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning,” *arXiv preprint arXiv:2309.16650*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.16650>
- [6] P. Liu, Z. Guo, M. Warke, S. Chintala, C. Paxton, N. M. M. Shafullah, and L. Pinto, “DynaMem: Online dynamic spatio-semantic memory for open world mobile manipulation,” *arXiv preprint arXiv:2411.04999*, 2024. [Online]. Available: <https://arxiv.org/abs/2411.04999>
- [7] M. A. Mdfaa, S. Lukina, T. Akhtyamov, A. Nigmatzyanov, D. Nalberskii, S. Zagoruyko, and G. Ferrer, “Spatiotemporal knowledge graphs as persistent scene memory for embodied question answering,” *arXiv preprint arXiv:2510.01483*, 2025, version 3, July 2026. [Online]. Available: <https://arxiv.org/abs/2510.01483v3>
- [8] H.-T. L. Chiang, Z. Xu, Z. Fu, M. G. Jacob *et al.*, “Mobility VLA: Multimodal instruction navigation with long-context VLMs and topological graphs,” *arXiv preprint arXiv:2407.07775*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.07775>
- [9] F. Li, J. Liao, C. Cheng, J. Wang, S. Chen, T. Wang, and W. Zhao, “Goal-oriented navigation instruction generation with tour video priors,” *arXiv preprint arXiv:2608.08596*, 2026. [Online]. Available: <https://arxiv.org/abs/2608.08596>
- [10] S. Bahl, R. Mendonca, L. Chen, U. Jain, and D. Pathak, “Affordances from human videos as a versatile representation for robotics,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 13 778–13 790. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2023/html/Bahl_Affordances_From_Human_Videos_as_a_Versatile_Representation_for_Robotics_CVPR_2023_paper.html
- [11] A. Bahety, A. Balaji, B. Abbatementte, and R. Martín-Martín, “SafeMimic: Towards safe and autonomous human-to-robot imitation for mobile manipulation,” in *Robotics: Science and Systems (RSS)*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.15847>
- [12] X. Xu, J. Park, H. Zhang, E. Cousineau, A. Bhat, J. Barreiros, D. Wang, J. Bohg, and S. Song, “HoMMI: Learning whole-body mobile manipulation from human demonstrations,” *arXiv preprint arXiv:2603.03243*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.03243>
- [13] M. Ahn, A. Brohan, N. Brown, Y. Chebotar *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022. [Online]. Available: <https://arxiv.org/abs/2204.01691>
- [14] R. Shah, A. Yu, Y. Zhu, Y. Zhu, and R. Martín-Martín, “BUMBLE: Unifying reasoning and acting with vision-language models for building-wide mobile manipulation,” *arXiv preprint arXiv:2410.06237*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.06237>
- [15] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as Policies: Language model programs for embodied control,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. [Online]. Available: <https://arxiv.org/abs/2209.07753>
- [16] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang *et al.*, “Inner monologue: Embodied reasoning through planning with language models,” in *Conference on Robot Learning (CoRL)*, 2023. [Online]. Available: <https://proceedings.mlr.press/v205/huang23c.html>
- [17] P. Zhi, Z. Zhang, Y. Zhao, M. Han, Z. Zhang, Z. Li, Z. Jiao, B. Jia, and S. Huang, “Closed-loop open-vocabulary mobile manipulation with GPT-4V,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025. [Online]. Available: <https://arxiv.org/abs/2404.10220>
- [18] R. Lu, Y. Wu, E. Kou, L. Fu, W. Xiao, A. Mandlikar, Y. Xu, G. Shi, K. Goldberg, A. Chen, M. Chowdhury, Y. Zhu, L. J. Fan, and G. Wang, “ASPIRE: Agentic /skills discovery for robotics,” *arXiv preprint arXiv:2607.00272*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.00272>
- [19] P. Liu, Y. Orru, J. Vakil, C. Paxton, N. M. M. Shafullah, and L. Pinto, “OK-Robot: What really matters in integrating open-knowledge models for robotics,” in *Robotics: Science and Systems (RSS)*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.12202>
- [20] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, “The marathon 2: A navigation system,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020. [Online]. Available: <https://arxiv.org/abs/2003.00368>
- [21] F. Zacharias, C. Borst, and G. Hirzinger, “Capturing robot workspace structure: representing robot capabilities,” in *2007 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2007, pp. 3229–3236.
- [22] N. Vahrenkamp, T. Asfour, and R. Dillmann, “Robot placement based on reachability inversion,” in *2013 IEEE International Conference on Robotics and Automation*, 2013, pp. 1970–1975.
- [23] K. Chai, H. Lee, and J. J. Lim, “N2M: Bridging navigation and manipulation by learning pose preference from rollout,” *arXiv preprint arXiv:2509.18671*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.18671>
- [24] S. Jauhri, J. Peters, and G. Chaitzaki, “Robot learning of mobile manipulation with reachability behavior priors,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 8399–8406, 2022.
- [25] T. Welschehold, C. Dornhege, and W. Burgard, “Learning mobile manipulation actions from human demonstrations,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 3196–3201.
- [26] OpenAI, “GPT-5.6 Sol model,” OpenAI API documentation, accessed: Sep. 15, 2026. [Online]. Available: <https://developers.openai.com/api/docs/models/gpt-5.6-sol>
- [27] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath *et al.*, “SAM 3: Segment anything with concepts,” *arXiv preprint arXiv:2511.16719*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.16719>
- [28] B. Han, Y.-W. Chao, E. Coumans, C. Eppner, B. Sundaralingam, J. Deng, S. Birchfield, and A. Murali, “GraspGen-X: Cross-embodiment 6-DOF diffusion-based grasping,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. [Online]. Available: <https://github.com/NVlabs/GraspGenX>
- [29] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, “VGGT: Visual geometry grounded transformer,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 5294–5306. [Online]. Available: <https://arxiv.org/abs/2503.11651>
- [30] K. Zakka, “Mink: Python inverse kinematics based on MuJoCo,” Software, 2026. [Online]. Available: <https://github.com/kevinzakka/mink>
- [31] B. Sundaralingam, S. K. S. Hari, A. Fishman, C. Garrett *et al.*, “cuRobo: Parallelized collision-free minimum-jerk robot motion generation,” *arXiv preprint arXiv:2310.17274*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.17274>
- [32] M. H. Anjaria, M. E. Erceyes, V. Ghatnekar, N. Navarkar, H. Etukuru, X. Jiang, K. Patel, D. Kabra, N. Wojno, R. A. Prayage, S. Chintala, L. Pinto, N. M. M. Shafullah, and Z. J. Cui, “YOR: Your own mobile manipulator for generalizable robotics,” *arXiv preprint arXiv:2602.11150*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.11150>
- [33] Project Aria Team at Meta, “Aria Gen 2: An advanced research device for egocentric AI research,” *arXiv preprint*, 2025, meta Reality Labs Research.
- [34] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox, “Contact-GraspNet: Efficient 6-DoF grasp generation in cluttered scenes,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021. [Online]. Available: <https://arxiv.org/abs/2103.14127>
- [35] M. Zhang, Y. Du, C. Wu, J. Zhou, Z. Qi, J. Ma, and B. Zhou, “ApexNav: An adaptive exploration strategy for zero-shot object navigation with target-centric semantic fusion,” *arXiv preprint arXiv:2504.14478*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.14478>
- [36] S. Y. Gadre, M. Wortsman, G. Ilharco, L. Schmidt, and S. Song, “CoWs on Pasture: Baselines and benchmarks for language-driven zero-shot object navigation,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. [Online]. Available: <https://arxiv.org/abs/2203.10421>